

```
!pip install linearmodels geopandas shapely openpyxl -q

import pandas as pd
import numpy as np
import json
import geopandas as gpd
from shapely.geometry import shape, Point
import statsmodels.api as sm
from linearmodels.panel import PanelOLS
import warnings

warnings.filterwarnings('ignore')
```

```
# A. Loading GeoJSON Grid safely
with open('delhi_grid.geojson', 'r') as f:
    grid_data = json.load(f)

features = []
for feat in grid_data['features']:
    geom = shape(feat['geometry'])
    props = feat['properties']
    features.append({'geometry': geom, **props})

gdf_grid = gpd.GeoDataFrame(features, crs="EPSG:4326")

# B. Loading Indian Government DUSIB Data
df_dusib = pd.read_excel('dusib_coordinates.xlsx')
geometry = [Point(xy) for xy in zip(df_dusib['Longitude'], df_dusib['Latitude'])]
gdf_dusib = gpd.GeoDataFrame(df_dusib, geometry=geometry, crs="EPSG:4326")

# C. Performing Spatial Join
spatial_join = gpd.sjoin(gdf_dusib, gdf_grid, how="inner", predicate="intersects")
official_slum_grids = spatial_join['grid_id'].unique()
print(f"Identified {len(official_slum_grids)} official DUSIB grid cells out of {len(gdf_grid)} total grids")

Identified 37 official DUSIB grid cells out of 78 total grids.
```

```
# 2. Load and merge Unified Panel Data
df_heat = pd.read_csv('Delhi_Panel_Heat_Unified.csv')
df_aod = pd.read_csv('Delhi_Panel_AOD_Unified.csv')
df_no2 = pd.read_csv('Delhi_Panel_NO2_Unified.csv')

for df in [df_heat, df_aod, df_no2]:
    df['date'] = pd.to_datetime(df['date'])

# Group by to ensure no duplicates from multi-satellite passes
heat_daily = df_heat.groupby(['grid_id', 'date'])['delta_t'].mean().reset_index()
aod_daily = df_aod.groupby(['grid_id', 'date'])['aod_level'].mean().reset_index()
no2_daily = df_no2.groupby(['grid_id', 'date'])['no2_level'].mean().reset_index()

# Merge Master Panel
master = pd.merge(heat_daily, aod_daily, on=['grid_id', 'date'], how='inner')
master = pd.merge(master, no2_daily, on=['grid_id', 'date'], how='inner')
master = master.dropna()

print(f"N = {len(master)} daily observations")

N = 11212 daily observations
```

```
# 3. Construct official interaction terms

# Flag grids as 1 if they contain an official slum, 0 otherwise
master['informal_official'] = master['grid_id'].isin(official_slum_grids).astype(int)

# Create the final interaction terms
master['aod_x_official'] = master['aod_level'] * master['informal_official']
master['no2_x_official'] = master['no2_level'] * master['informal_official']
```

```
# Set Panel Index
panel_data = master.set_index(['grid_id', 'date'])
```

```
# 4. Regression (Two Way Fixed Effect)
```

```
print("FINAL TABLE 1: AOD THERMAL BLANKET (Official DUSIB Data)")
exog_aod = sm.add_constant(panel_data[['aod_level', 'aod_x_official']])
res_aod = PanelOLS(panel_data['delta_t'], exog_aod, entity_effects=True, time_effects=True, drop_absorbed=True)
print(res_aod.summary.tables[1])

print("\n" + "="*60)
print("FINAL TABLE 2: NO2 COMBUSTION PENALTY (Official DUSIB Data)")
print("="*60)
exog_no2 = sm.add_constant(panel_data[['no2_level', 'no2_x_official']])
res_no2 = PanelOLS(panel_data['delta_t'], exog_no2, entity_effects=True, time_effects=True, drop_absorbed=True)
print(res_no2.summary.tables[1])
```

FINAL TABLE 1: AOD THERMAL BLANKET (Official DUSIB Data)
Parameter Estimates

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
const	18.562	0.1375	134.96	0.0000	18.292	18.832
aod_level	-1.0220	0.1249	-8.1814	0.0000	-1.2668	-0.7771
aod_x_official	-0.1992	0.0514	-3.8773	0.0001	-0.2999	-0.0985

FINAL TABLE 2: NO2 COMBUSTION PENALTY (Official DUSIB Data)

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
const	17.949	0.1056	170.01	0.0000	17.742	18.156
no2_level	-5127.2	1760.7	-2.9121	0.0036	-8578.5	-1676.0
no2_x_official	-2703.9	1537.1	-1.7591	0.0786	-5716.9	309.02

✧ Exploratory Data Analysis & Spatial Heat Maps

```
!pip install geopandas shapely matplotlib seaborn -q

import geopandas as gpd
import matplotlib.pyplot as plt
import seaborn as sns
from shapely.geometry import Point, shape
import json
import warnings
warnings.filterwarnings('ignore')

# 1. Physical boundary of Delhi
with open('delhi_grid.geojson', 'r') as f:
    grid_data = json.load(f)
features = []
for feat in grid_data['features']:
    geom = shape(feat['geometry'])
    props = feat['properties']
    features.append({'geometry': geom, **props})
gdf_grid = gpd.GeoDataFrame(features, crs="EPSG:4326")

# 2. Lat and Long for Informal settlements
df_dusib = pd.read_excel('dusib_coordinates.xlsx')
geometry = [Point(xy) for xy in zip(df_dusib['Longitude'], df_dusib['Latitude'])]
gdf_dusib = gpd.GeoDataFrame(df_dusib, geometry=geometry, crs="EPSG:4326")

# 3. Heat and Pollution Data
df_heat = pd.read_csv('Delhi_Panel_Heat_Unified.csv')
df_aod = pd.read_csv('Delhi_Panel_AOD_Unified.csv')
df_no2 = pd.read_csv('Delhi_Panel_NO2_Unified.csv')
```

```

for df in [df_heat, df_aod, df_no2]:
    df['date'] = pd.to_datetime(df['date'])

heat_daily = df_heat.groupby(['grid_id', 'date'])['delta_t'].mean().reset_index()
aod_daily = df_aod.groupby(['grid_id', 'date'])['aod_level'].mean().reset_index()
no2_daily = df_no2.groupby(['grid_id', 'date'])['no2_level'].mean().reset_index()

master = pd.merge(heat_daily, aod_daily, on=['grid_id', 'date'], how='inner')
master = pd.merge(master, no2_daily, on=['grid_id', 'date'], how='inner')
master = master.dropna()

```

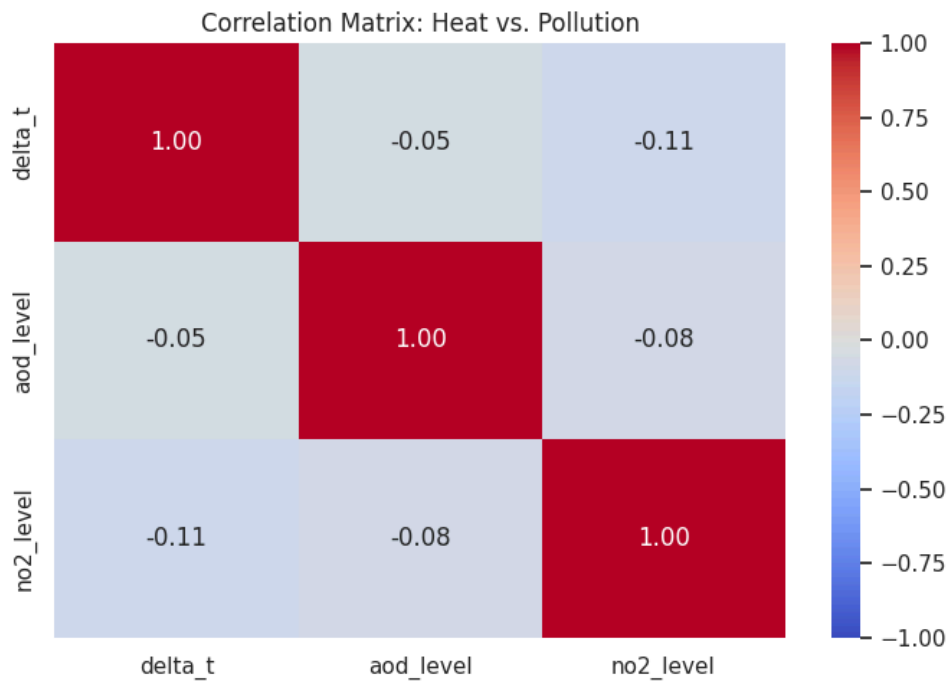
Correlational Matrix

```

sns.set_theme(style="whitegrid")

plt.figure(figsize=(7, 5))
corr = master[['delta_t', 'aod_level', 'no2_level']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1, fmt=".2f")
plt.title('Correlation Matrix: Heat vs. Pollution')
plt.tight_layout()
plt.show()

```



Distributions

```

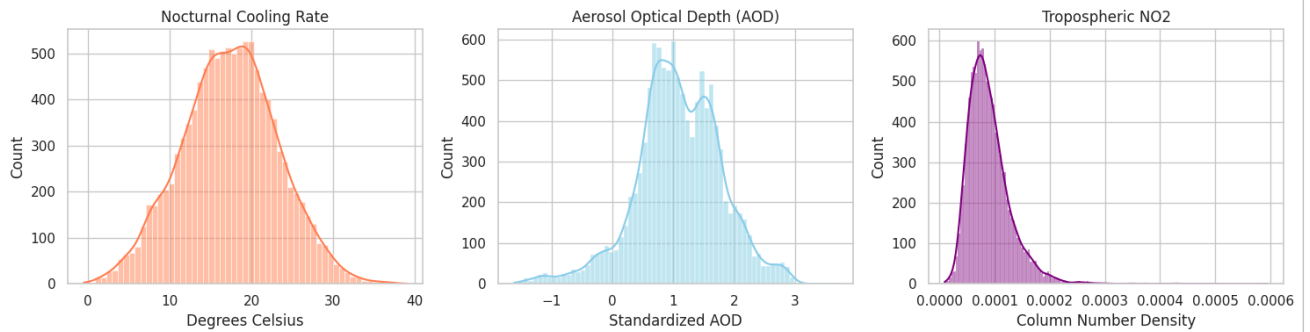
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.histplot(master['delta_t'], kde=True, ax=axes[0], color='coral')
axes[0].set_title('Nocturnal Cooling Rate')
axes[0].set_xlabel('Degrees Celsius')

sns.histplot(master['aod_level'], kde=True, ax=axes[1], color='skyblue')
axes[1].set_title('Aerosol Optical Depth (AOD)')
axes[1].set_xlabel('Standardized AOD')

sns.histplot(master['no2_level'], kde=True, ax=axes[2], color='purple')
axes[2].set_title('Tropospheric NO2')
axes[2].set_xlabel('Column Number Density')

plt.tight_layout()
plt.show()

```



Geographic Spatial heatmap

```
# Aggregate values spatially by grid to find the 3-year average per location
spatial_agg = master.groupby('grid_id')[['delta_t', 'aod_level', 'no2_level']].mean().reset_index()
gdf_mapped = gdf_grid.merge(spatial_agg, on='grid_id', how='left')

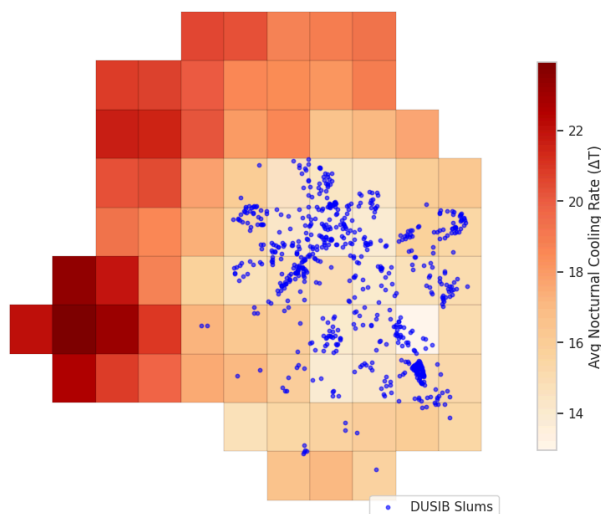
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# Map 1: Cooling Rate + Slums
gdf_mapped.plot(column='delta_t', cmap='OrRd', legend=True,
                 legend_kws={'label': 'Avg Nocturnal Cooling Rate (\u0394T)', 'shrink': 0.7},
                 ax=axes[0], missing_kws={'color': 'lightgrey'}, edgecolor='black', linewidth=0.1)
gdf_dusib.plot(ax=axes[0], color='blue', markersize=10, alpha=0.6, label='DUSIB Slums')
axes[0].set_title('Spatial Distribution: Nocturnal Cooling & Slum Locations', fontsize=14)
axes[0].axis('off')
axes[0].legend(loc='lower right')

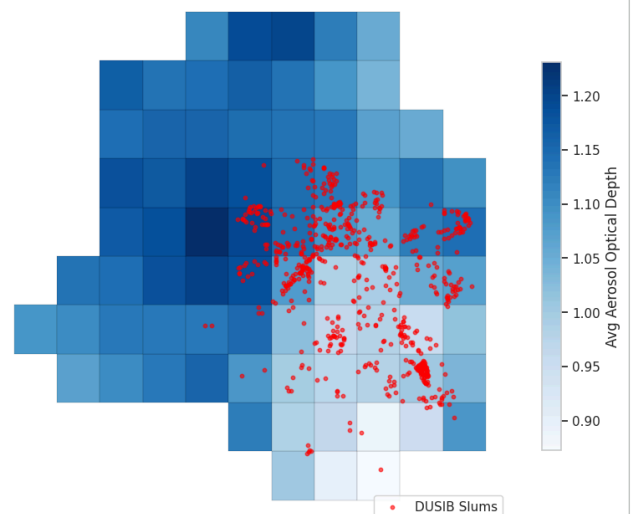
# Map 2: AOD + Slums
gdf_mapped.plot(column='aod_level', cmap='Blues', legend=True,
                 legend_kws={'label': 'Avg Aerosol Optical Depth', 'shrink': 0.7},
                 ax=axes[1], missing_kws={'color': 'lightgrey'}, edgecolor='black', linewidth=0.1)
gdf_dusib.plot(ax=axes[1], color='red', markersize=10, alpha=0.6, label='DUSIB Slums')
axes[1].set_title('Spatial Distribution: Aerosol Pollution & Slum Locations', fontsize=14)
axes[1].axis('off')
axes[1].legend(loc='lower right')

plt.tight_layout()
plt.show()
```

Spatial Distribution: Nocturnal Cooling & Slum Locations



Spatial Distribution: Aerosol Pollution & Slum Locations



Interactive spatial maps

```

!pip install folium geopandas -q

import geopandas as gpd
import folium
import json

# 1. Loading DUSIB Data
df_dusib = pd.read_excel('dusib_coordinates.xlsx')

# 2. Initializing the Interactive Map centered on Delhi
delhi_center = [28.6139, 77.2090]
m = folium.Map(location=delhi_center, zoom_start=10, tiles='CartoDB positron')

# 3. Adding the Spatial Master Grid
with open('delhi_grid.geojson', 'r') as f:
    grid_data = json.load(f)

folium.GeoJson(
    grid_data,
    name="Research Grid (3.5 x 5.5 km)",
    style_function=lambda feature: {
        'fillColor': '#3388ff',
        'color': 'black',
        'weight': 1,
        'fillOpacity': 0.05,
    },
    tooltip=folium.GeoJsonTooltip(fields=['grid_id'], aliases=['Grid ID:'])
).add_to(m)

# 4. Adding DUSIB Slum Markers
slum_group = folium.FeatureGroup(name="Official DUSIB Clusters")

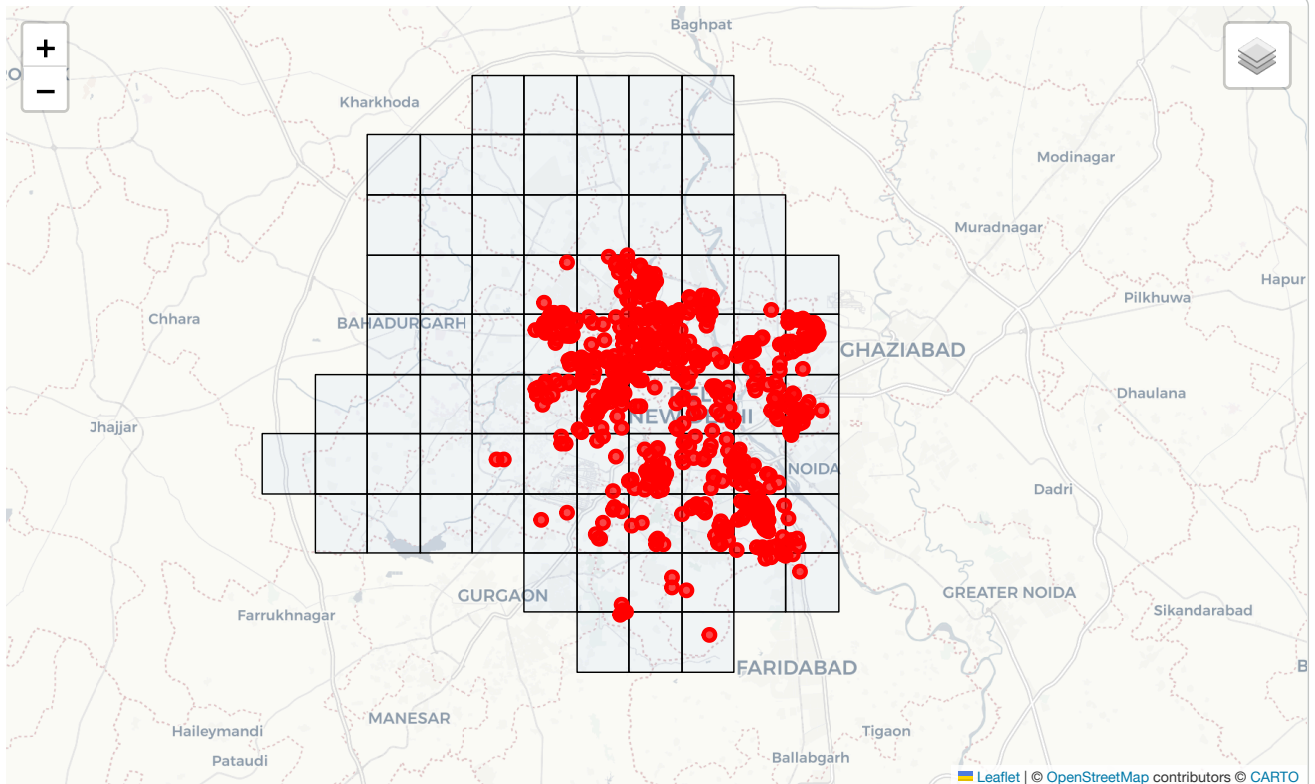
for idx, row in df_dusib.iterrows():
    folium.CircleMarker(
        location=[row['Latitude'], row['Longitude']],
        radius=4, # Size of the dot
        popup=f"<b>Location:</b> {row.get('Location', 'Unknown')}<br><b>Lat:</b> {row['Latitude']}<br><b>Lo",
        tooltip="Click for details",
        color='red',
        fill=True,
        fill_color='red',
        fill_opacity=0.7
    ).add_to(slum_group)

slum_group.add_to(m)

# 5. Adding a Layer Control
folium.LayerControl().add_to(m)

m.save("Interactive_Slum_Map.html")
m

```



Spatial Vulnerability Ranking (Identifying the Worst-Hit Slums)

```
import pandas as pd
import numpy as np
import geopandas as gpd
from shapely.geometry import Point, shape
import json
import warnings

warnings.filterwarnings('ignore')

with open('delhi_grid.geojson', 'r') as f:
    grid_data = json.load(f)
features = []
for feat in grid_data['features']:
    geom = shape(feat['geometry'])
    props = feat['properties']
    features.append({'geometry': geom, **props})
gdf_grid = gpd.GeoDataFrame(features, crs="EPSG:4326")

df_dusib = pd.read_excel('dusib_coordinates.xlsx')
geometry = [Point(xy) for xy in zip(df_dusib['Longitude'], df_dusib['Latitude'])]
gdf_dusib = gpd.GeoDataFrame(df_dusib, geometry=geometry, crs="EPSG:4326")

spatial_join = gpd.sjoin(gdf_dusib, gdf_grid, how="inner", predicate="intersects")

df_heat = pd.read_csv('Delhi_Panel_Heat_Unified.csv')
df_aod = pd.read_csv('Delhi_Panel_AOD_Unified.csv')

# Calculating the 3-year pre-monsoon average for each grid
heat_avg = df_heat.groupby('grid_id')[['delta_t']].mean().reset_index()
aod_avg = df_aod.groupby('grid_id')[['aod_level']].mean().reset_index()
grid_agg = pd.merge(heat_avg, aod_avg, on='grid_id', how='inner')

# Calculating the composite vulnerability score
grid_agg['z_heat'] = (grid_agg['delta_t'] - grid_agg['delta_t'].mean()) / grid_agg['delta_t'].std()
grid_agg['z_aod'] = (grid_agg['aod_level'] - grid_agg['aod_level'].mean()) / grid_agg['aod_level'].std()
grid_agg['composite_penalty'] = grid_agg['z_aod'] - grid_agg['z_heat']

slum_vulnerability = pd.merge(spatial_join, grid_agg, on='grid_id', how='inner')
```

```
print("Top 10 most affected informal settlements (High Smog + Low Cooling)")
worst_compound = slum_vulnerability.sort_values(by='composite_penalty', ascending=False)
print(worst_compound[['Location', 'grid_id', 'composite_penalty', 'delta_t', 'aod_level']].head(10).to_string(index=False))
```

Top 10 most affected informal settlements (High Smog + Low Cooling)

	Location	grid_id	composite_penalty	delta_t	aod_level
	JJ Camp Indira Camp No-II Vikas Puri	7708,2863	2.509588	14.680617	0.901549
	Krishna Park Vikas Puri Indira Camp No. 4	7708,2863	2.509588	14.680617	0.901549
	Shanker Garden B-Block Vikas Puri	7708,2863	2.509588	14.680617	0.901549
	Double Story Sweeper Tenements Tilak Nagar	7708,2863	2.509588	14.680617	0.901549
	Rajiv Gandhi Camp Along Nalla Near Shamshan Bhumi Has	7708,2863	2.509588	14.680617	0.901549
	JJC A-562, Pankha Road Near Nalla, Uttam Nagar	7708,2863	2.509588	14.680617	0.901549
	Indira Camp No.6 Vikas Puri	7708,2863	2.509588	14.680617	0.901549
	Near Shamshan Bhumi Hastal	7708,2863	2.509588	14.680617	0.901549
	Indira Camp No.-5 Vikas Puri	7708,2863	2.509588	14.680617	0.901549
	Indira Colony Chaukhandi Road Opposite Yog Mandir and	7708,2863	2.509588	14.680617	0.901549

```
print(f"Top 5 worst by pure lack of cooling (Lowest delta t)")
print(worst_compound.sort_values(by='delta_t')[['Location', 'delta_t', 'grid_id']].head(5).to_string(index=False))
```

Top 5 worst by pure lack of cooling (Lowest delta t)

	Location	delta_t	grid_id
	T-Huts Behind MCD School Village Sarai Kale Khan	12.872181	7728,2858
	Indira Gandhi Tea Market Shriniwas Puri	12.872181	7728,2858
	Indira Gandhi Camp No-3 near Railway Line Hari Nagar As	12.872181	7728,2858
	JJC Indira Gandhi Camp Shriniwas Puri	12.872181	7728,2858
	Chander Arya Vidya Mandir	12.872181	7728,2858

```
print("Top 5 worst by pure smog (Highest AOD)")
print(worst_compound.sort_values(by='aod_level', ascending=False)[['Location', 'aod_level', 'grid_id']].head(5).to_string(index=False))
```

Top 5 worst by pure smog (Highest AOD)

	Location	aod_level	grid_id
	Hans Raj Mulk Raj Bhatta Jwalapuri	0.907865	7708,2868
	F-Block Mangolpuri	0.907865	7708,2868
	JJ Camp Indira Camp No-II Vikas Puri	0.901549	7708,2863
	Double Story Sweeper Tenements Tilak Nagar	0.901549	7708,2863
	Rajiv Gandhi Camp Along Nalla Near Shamshan Bhumi Has	0.901549	7708,2863

Interactive Geographic Vulnerability Map

```
!pip install folium geopandas -q

import geopandas as gpd
from shapely.geometry import Point, shape
import folium
import json

# 1. Load Data
with open('delhi_grid.geojson', 'r') as f:
    grid_data = json.load(f)
features = []
for feat in grid_data['features']:
    features.append({'geometry': shape(feat['geometry']), **feat['properties']})
gdf_grid = gpd.GeoDataFrame(features, crs="EPSG:4326")

df_dusib = pd.read_excel('dusib_coordinates.xlsx')
geometry = [Point(xy) for xy in zip(df_dusib['Longitude'], df_dusib['Latitude'])]
gdf_dusib = gpd.GeoDataFrame(df_dusib, geometry=geometry, crs="EPSG:4326")

df_heat = pd.read_csv('Delhi_Panel_Heat_Unified.csv')
df_aod = pd.read_csv('Delhi_Panel_AOD_Unified.csv')

# 2. Process Vulnerability Scores
heat_avg = df_heat.groupby('grid_id')[['delta_t']].mean().reset_index()
aod_avg = df_aod.groupby('grid_id')[['aod_level']].mean().reset_index()
grid_agg = pd.merge(heat_avg, aod_avg, on='grid_id', how='inner')
```

```

grid_agg['z_heat'] = (grid_agg['delta_t'] - grid_agg['delta_t'].mean()) / grid_agg['delta_t'].std()
grid_agg['z_aod'] = (grid_agg['aod_level'] - grid_agg['aod_level'].mean()) / grid_agg['aod_level'].std()
grid_agg['composite_penalty'] = grid_agg['z_aod'] - grid_agg['z_heat']

slum_vulnerability = pd.merge(gpd.sjoin(gdf_dusib, gdf_grid, how="inner", predicate="intersects"), grid_agg)

# Identify the worst locations
worst_compound = slum_vulnerability.sort_values(by='composite_penalty', ascending=False).head(15)
worst_heat = slum_vulnerability.sort_values(by='delta_t').head(15)
worst_smog = slum_vulnerability.sort_values(by='aod_level', ascending=False).head(15)

# 3. Build the Map
m = folium.Map(location=[28.6139, 77.2090], zoom_start=11, tiles='CartoDB positron')

# Add all slums as tiny background dots
all_slums = folium.FeatureGroup(name="All DUSIB Slums")
for idx, row in slum_vulnerability.iterrows():
    folium.CircleMarker(location=[row['Latitude'], row['Longitude']], radius=2, color='grey', fill=True,
all_slums.add_to(m)

# Add Ground Zeros (Compound Penalty)
compound_group = folium.FeatureGroup(name="Ground Zeros (Compound Penalty)")
for idx, row in worst_compound.iterrows():
    folium.CircleMarker(
        location=[row['Latitude'], row['Longitude']], radius=8, color='purple', fill=True, fill_opacity=0
        popup=f"<b>{row['Location']}</b><br>Cooling: {row['delta_t']:.2f}°C<br>Smog (AOD): {row['aod_level']:.2f}"
    ).add_to(compound_group)
compound_group.add_to(m)

# Add Heat Traps
heat_group = folium.FeatureGroup(name="Severe Heat Traps")
for idx, row in worst_heat.iterrows():
    folium.CircleMarker(
        location=[row['Latitude'], row['Longitude']], radius=8, color='red', fill=True, fill_opacity=0.9,
        popup=f"<b>{row['Location']}</b><br>Cooling: {row['delta_t']:.2f}°C<br>Smog (AOD): {row['aod_level']:.2f}"
    ).add_to(heat_group)
heat_group.add_to(m)

# Add Smog Zones
smog_group = folium.FeatureGroup(name="Maximum Smog Zones")
for idx, row in worst_smog.iterrows():
    folium.CircleMarker(
        location=[row['Latitude'], row['Longitude']], radius=8, color='blue', fill=True, fill_opacity=0.9,
        popup=f"<b>{row['Location']}</b><br>Cooling: {row['delta_t']:.2f}°C<br>Smog (AOD): {row['aod_level']:.2f}"
    ).add_to(smog_group)
smog_group.add_to(m)

folium.LayerControl().add_to(m)
m.save("Vulnerability_Map.html")
m

```